Grocery Management System

Course: Visual Programming Lab

Submitted By:

Samra Ramzan

232201009

CS-5A

Submitted To:

Sir Uzair Hassan

Department of Computer Science

Kahuta Institute of Space and Technology

April 19, 2026

Contents

1 Project Introduction **2**

2 OOP Concepts Used **2**

2.1 Encapsulation 2

2.2 Inheritance 2

2.3 Polymorphism 3

2.4 Composition 3

3 Admin Module **4**

4 Cashier Module **5**

5 Customer Module **6**

6 Database Integration **7**

6.1 Database Collections 7

7 Conclusion **8**

1 Project Introduction

Manual grocery management systems are inefficient and error-prone. This system automates billing, inventory tracking, and user role management using object-oriented programming principles. The application provides separate modules for Admin, Cashier, and Customer roles, ensuring a streamlined workflow for grocery stores and supermarkets.

2 OOP Concepts Used

2.1 Encapsulation

Data hiding is achieved through properties with getters and setters. Internal state is protected from direct manipulation.

```
class Product {
    public int Id { get; set; }
    public string Name { get; set; }
    private decimal price;
    public decimal Price {
        get { return price; }
        set { if(value > 0) price = value; }
    }
}
```

2.2 Inheritance

The User class serves as a base class, while specific user types inherit common functionality.

```
class User {
    public string Username { get; set; }
    public string Password { get; set; }
    public virtual void Menu() {}
}
class Admin : User {
    public override void Menu() {
        Console.WriteLine("Admin Menu");
    }
}
class Cashier : User {
    public override void Menu() {
        Console.WriteLine("Cashier Menu");
    }
}
```

```
class Customer : User {
    public override void Menu() {
        Console.WriteLine("Customer Menu");
    }
}
```

2.3 Polymorphism

Method overriding allows different implementations of the same method across derived classes.

```
public virtual void DisplayRole() {
    Console.WriteLine("General User");
}
class Admin : User {
    public override void DisplayRole() {
        Console.WriteLine("Administrator");
    }
}
```

2.4 Composition

Classes contain instances of other classes to build complex functionality.

```
class Bill {
    public int BillId { get; set; }
    public DateTime Date { get; set; }
    public List<CartItem> Items { get; set; }
    public decimal TotalAmount { get; set; }
}
class CartItem {
    public Product Product { get; set; }
    public int Quantity { get; set; }
    public decimal SubTotal { get; set; }
}
```

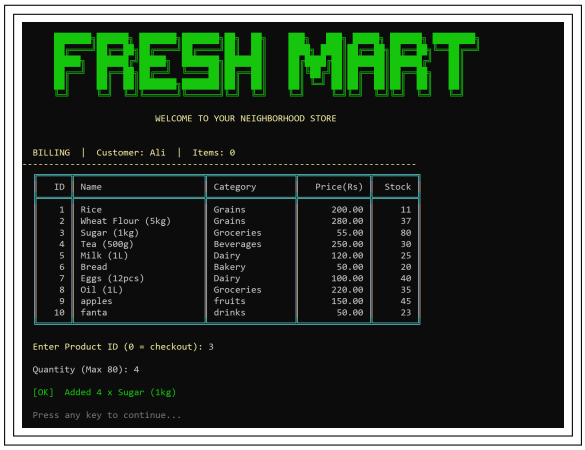
3 Admin Module

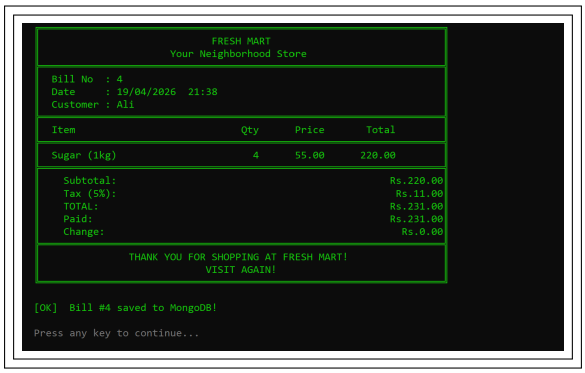
The Admin module provides full control over the grocery management system. Administrators can manage products, users, and view all transactions.

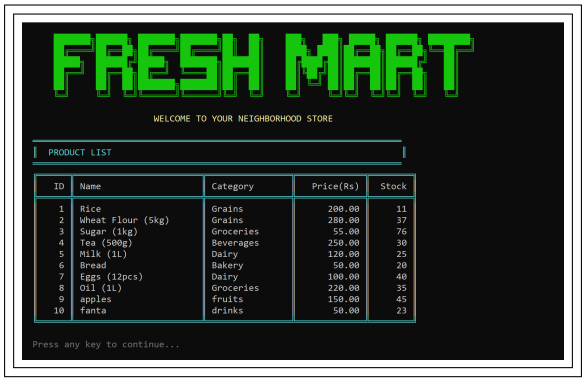
Figure: Admin Dashboard and Management Screens

4 Cashier Module

The Cashier module handles billing operations, product search, and sales transactions. Cashiers can process customer purchases and generate receipts.







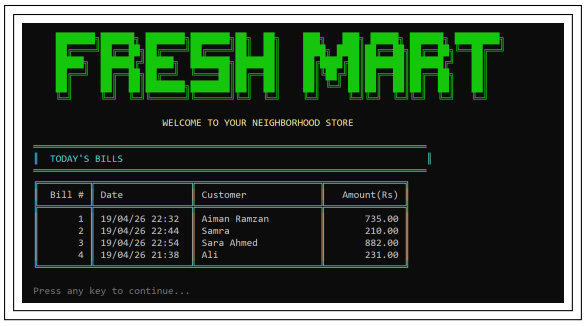


Figure: Cashier Billing and Transaction Screens

5 Customer Module

The Customer module allows users to browse products, add items to cart, and place orders. Customers can view their order history and track purchases.

Figure: Customer Shopping Interface (Part 1)

Figure: Customer Shopping Interface (Part 2)

6 Database Integration

The system uses MongoDB for data persistence, providing flexible document-based storage for products, users, and transactions.

```
using MongoDB.Driver;

// Database connection setup
var client = new MongoClient("mongodb://localhost:27017");
var database = client.GetDatabase("GroceryDB");
var productsCollection = database.GetCollection<Product>("Products");
var usersCollection = database.GetCollection<User>("Users");
var billsCollection = database.GetCollection<Bill>("Bills");

// CRUD Operations Example
public async Task AddProduct(Product product) {
    await productsCollection.InsertOneAsync(product);
}

public async Task<List<Product>> GetAllProducts() {
    return await productsCollection.Find(_ => true).ToListAsync();
}
```

6.1 Database Collections

- **Products:** Stores product ID, name, price, quantity, and category
- **Users:** Stores user credentials, roles (Admin/Cashier/Customer), and profile info
- **Bills:** Stores transaction details, items purchased, total amount, and timestamps
- **Carts:** Temporary storage for customer shopping carts

7 Conclusion

The Grocery Management System successfully implements key object-oriented programming concepts including Encapsulation, Inheritance, Polymorphism, and Composition. The integration with MongoDB provides reliable data persistence, while the three-tier user architecture (Admin, Cashier, Customer) ensures appropriate access control and functionality separation.

Key Achievements:

- Fully automated billing and inventory management
- Role-based access control with secure authentication
- Real-time product search and cart management
- Persistent data storage using MongoDB
- User-friendly graphical interface

Future enhancements may include barcode scanning integration, sales analytics dashboard, and online payment gateway integration.

— End of Report —